Document: WG21 P0613R0
Audience: WG21
Author: William M. (Mike) Miller
Edison Design Group, Inc.
wmm@edg.com
Date: 2017-02-27

NB comment GB15: Resolution of Core Issue 2011

2011. Unclear effect of reference capture of reference

Section: 5.1.5 [expr.prim.lambda] **Status:** review **Submitter:** Ville Voutilainen **Date:** 2014-09-28 **Drafting:** Miller

(From messages 25953, 25956, 25958 through 25960, 25962, and 25964.)

The Standard refers to capturing "entities," and a reference is an entity. However, it is not clear what capturing a reference by reference would mean. In particular, 5.1.5 [expr.prim.lambda] paragraph 16 says,

> It is unspecified whether additional unnamed non-static data members are declared in the closure type for entities captured by reference.

If a reference captured by reference is not represented by a member, it is hard to see how something like the following example could work:

```
#include <functional>
#include <iostream>

std::function<void()> make_function(int& x) {
  return [&]{ std::cout << x << std::endl; };
}

int main() {
  int i = 3;
  auto f = make_function(i);
  i = 5;
  f();
}
```

Should this be undefined behavior or should it print `5`?

Proposed resolution (November, 2014) [SUPERSEDED]:

1. Change 5.1.5 [expr.prim.lambda] paragraph 18 as follows:

> Every *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use (3.2 [basic.def.odr]) of an entity captured by copy is transformed into an access to the corresponding unnamed data member of the closure type. [*Note:* An *id-expression* that is not an odr-use refers to the original entity, never to a member of the closure type. Furthermore, such an *id-expression* does not cause the implicit capture of the entity. —*end note*] If `this` is captured, each odr-use of `this` is transformed into an access to the corresponding unnamed data member of the closure type, cast (5.4 [expr.cast]) to the type of `this`. [*Note:* The cast ensures that the transformed expression is a prvalue. —*end note*] **An *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use of a reference captured by reference refers to the entity to which the captured reference is bound and not to the captured reference. [*Note:* Such odr-uses are not invalidated by the end of the captured reference's lifetime. —*end note*]** [*Example:*

```
void f(const int*);
void g() {
  const int N = 10;
  [=] {
  int arr[N]; // OK: not an odr-use, refers to automatic variable
  f(&N);      // OK: causes N to be captured; &N points to the
              // corresponding member of the closure type
  };
}
auto h(int &r) {
  return [&]() {
    ++r;      // Valid after h returns if the lifetime of the
              // object to which r is bound has not ended
```

```
  };
}
```

—*end example*]

2. Change 5.1.5 [expr.prim.lambda] paragraph 23 as follows:

[*Note:* If ~~an~~ **a non-reference** entity is implicitly or explicitly captured by reference, invoking the function call operator of the corresponding *lambda-expression* after the lifetime of the entity has ended is likely to result in undefined behavior. —*end note*]

Proposed resolution (February, 2017):

1. Change 5.1.5 [expr.prim.lambda] paragraph 17 as follows:

Every *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use (3.2 [basic.def.odr]) of an entity captured by copy is transformed into an access to the corresponding unnamed data member of the closure type. [*Note:* An *id-expression* that is not an odr-use refers to the original entity, never to a member of the closure type. Furthermore, such an *id-expression* does not cause the implicit capture of the entity. —*end note*] If `*this` is captured by copy, each odr-use of `this` is transformed into a pointer to the corresponding unnamed data member of the closure type, cast (5.4 [expr.cast]) to the type of `this`. [*Note:* The cast ensures that the transformed expression is a prvalue. —*end note*] **An *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use of a reference captured by reference refers to the entity to which the captured reference is bound and not to the captured reference. [*Note:* The validity of such captures is determined by the lifetime of the object to which the reference refers, not by the lifetime of the reference itself. —*end note*]** [*Example:*

```
void f(const int*);
void g() {
  const int N = 10;
  [=] {
  int arr[N]; // OK: not an odr-use, refers to automatic variable
  f(&N);      // OK: causes N to be captured; &N points to the
              // corresponding member of the closure type
  };
}
auto h(int &r) {
  return [&] {
    ++r;      // Valid after h returns if the lifetime of the
              // object to which r is bound has not ended
  };
}
```

—*end example*]

2. Change 5.1.5 [expr.prim.lambda] paragraph 25 as follows:

[*Note:* If ~~an~~ **a non-reference** entity is implicitly or explicitly captured by reference, invoking the function call operator of the corresponding *lambda-expression* after the lifetime of the entity has ended is likely to result in undefined behavior. —*end note*]